UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO - UNIDAD 4

FNBC y desnormalización controlada en Food Store

Proyecto integrador: Food Store
Forma Normal de Boyce-Codd · Optimización medida

Materia: Base de Datos II
Carrera: Tecnicatura Universitaria en Programación (UTN)

Autores:
Lemus Nahuel
Anglat Juan
Elias Juan Cruz

Año 2026

1. Parte 1 - Dependencias, claves y FNBC

Se parte de R(LoteID, DepositoID, ResponsableControlID). La extensión se creó en el esquema aislado tp_u4, porque las tablas maestras lote, deposito y usuario no existían en el Food Store vigente.

Dependencia funcional	Fundamento
(LoteID, DepositoID) → ResponsableControlID	Para cada lote y depósito interviene un responsable único.
ResponsableControlID → DepositoID	Cada responsable pertenece a un único depósito.

Clausuras relevantes

Conjunto X	Cierre(X)	¿Superclave?
{L}	{L}	No
{D}	{D}	No
{R}	{R,D}	No
{L,D}	{L,D,R}	Sí, clave candidata
{L,R}	{L,R,D}	Sí, clave candidata
{D,R}	{D,R}	No

L es obligatorio en toda clave porque ninguna dependencia lo produce. Con L+D se obtiene R; con L+R se obtiene D. Por lo tanto, el conjunto completo de claves candidatas es {L,D} y {L,R}. Los tres atributos son primos; no hay atributos no primos.

Violación de FNBC y anomalías

La dependencia R → D es no trivial y cierre(R)={R,D}, que no contiene L. Su determinante no es superclave; por la definición formal, la relación original no está en FNBC. Aunque todos los atributos son primos y la 3FN admite esa dependencia, la FNBC no la admite.

Anomalía	Escenario concreto sobre la instancia
Inserción	No se puede registrar que el responsable 803 pertenece al depósito 30 hasta que controle algún lote: LoteID es obligatorio en la tabla original.
Borrado	Al borrar (503,31,802), se pierde el único dato que indica que 802 pertenece al depósito 31.
Actualización	Si 801 pasa del depósito 30 al 31, hay que cambiar (501,30,801) y (502,30,801); actualizar una sola fila deja información contradictoria.

Descomposición sin pérdida

Se descompone por R → D en ResponsableDeposito(R,D) y ControlLoteAlmacen(L,R). La intersección es {R}; R es clave de ResponsableDeposito. Por el criterio de superclave sobre el atributo común, la reunión natural es sin pérdida. Ambas tablas quedan en FNBC. La dependencia original LD → R no se preserva solo con estas proyecciones; un trigger la hace cumplir al impedir dos responsables para el mismo lote y depósito.

```
R1(R,D): PRIMARY KEY (ResponsableControlID)
R2(L,R): PRIMARY KEY (LoteID, ResponsableControlID)
Vista: R2 JOIN R1 USING (ResponsableControlID)
```

Migración comprobada: la vista reconstruyó las tres tuplas originales y el EXCEPT ALL bidireccional devolvió 0 filas. El intento de asignar al lote 501 un segundo responsable del depósito 30 fue rechazado.

2. Parte 2 - Top de categorías con datos precalculados

Adaptación al esquema real

La consulta de la consigna usa dp.subtotal, dp.eliminado y ped.eliminado, que no existen en el modelo entregado. Se usa dp.cantidad × dp.precio_unitario y se incluyen las ventas históricas. Como pedido.fecha es TIMESTAMPTZ, el día se delimita con un rango en la zona America/Argentina/Buenos_Aires. Se agrupa por ID y nombre de categoría para no fusionar categorías homónimas.

```
WHERE ped.fecha >= CURRENT_DATE::timestamptz
  AND ped.fecha < (CURRENT_DATE + 1)::timestamptz
SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
```

Elección y sincronización

Se eligieron columnas precalculadas con triggers: fecha_venta_u4, categoria_id_u4, categoria_nombre_u4 y subtotal_u4 en detalle_pedido. El plan inicial recorre 20.007 detalles y encadena tres Hash Join para obtener 666 líneas del día. La consulta nueva lee solo detalle_pedido con un índice por fecha. Los triggers recalculan la línea al insertarla o modificar sus valores y propagan cambios de fecha del pedido, categoría del producto y nombre de categoría en la misma transacción. Por ello, al confirmar la escritura, la redundancia corresponde a la fuente; ROLLBACK revierte ambos cambios. Las cuatro columnas, los triggers y el índice se pueden retirar sin perder ventas, porque cantidad, precio_unitario, pedido_id y producto_id siguen siendo la fuente de verdad.

Cambio en fuente	Mecanismo de sincronización
INSERT/UPDATE de línea	BEFORE trigger calcula día, categoría, nombre y subtotal.
UPDATE pedido.fecha	AFTER trigger actualiza el día de todas sus líneas.
UPDATE producto.categoria_id	AFTER trigger actualiza ID y nombre de categoría de sus líneas.
UPDATE categoria.nombre	AFTER trigger actualiza el nombre almacenado en sus líneas.

Consulta que usa la estructura desnormalizada

```
SELECT categoria_id_u4, categoria_nombre_u4 AS categoria,
       SUM(subtotal_u4)::numeric(14,2) AS total_vendido
FROM detalle_pedido WHERE fecha_venta_u4 = CURRENT_DATE
GROUP BY categoria_id_u4, categoria_nombre_u4
ORDER BY total_vendido DESC, categoria_id_u4 LIMIT 5;
```

Comparación medida con EXPLAIN (ANALYZE, BUFFERS)

Indicador	Antes: 4 tablas	Después: 1 tabla
Execution Time (captura)	14,440 ms	1,280 ms
Mediana, 5 repeticiones	14,576 ms	1,230 ms
Acceso que domina	Hash Join + Seq Scan de detalle	Bitmap Heap Scan + Bitmap Index Scan
Filas del top 5	5	5, exactamente iguales

En esta copia local la mediana fue aproximadamente 11,9 veces menor. La mejora medida es de lectura: la redundancia ocupa más espacio y cada cambio de pedido, producto o categoría puede actualizar varias líneas. Los tiempos no se extrapolan a Railway.

3. Evidencia del motor, auditoría y reproducción

Entorno de prueba

Copia local poblada con el esquema, datos, restricciones e índices previos en PostgreSQL 18.3 embebido mediante PGlite: 20 categorías, 1.000 productos, 203 clientes, 20.005 pedidos y 20.007 detalles. Se ejecutó ANALYZE antes de medir y tras cargar las columnas. El día probado tuvo 666 líneas. Las salidas siguientes son extractos literales de EXPLAIN ANALYZE; los scripts entregados contienen ambas consultas completas.

Plan anterior - cuatro tablas

```
HashAggregate (actual time=14.267..14.304 rows=20.00 loops=1)
  -> Hash Join (actual time=1.639..13.768 rows=666.00 loops=1)
     -> Hash Join (actual time=1.583..13.295 rows=666.00 loops=1)
        -> Hash Join (actual time=0.816..12.021 rows=666.00 loops=1)
           -> Seq Scan on detalle_pedido dp
              (actual time=0.006..5.141 rows=20007.00 loops=1)
Buffers: shared hit=329
Planning Time: 0.899 ms
Execution Time: 14.440 ms
```

Plan posterior - columnas precalculadas

```
HashAggregate (actual time=1.168..1.184 rows=20.00 loops=1)
  -> Bitmap Heap Scan on detalle_pedido
     (actual time=0.135..0.639 rows=666.00 loops=1)
     Recheck Cond: (fecha_venta_u4 = CURRENT_DATE)
     -> Bitmap Index Scan on idx_detalle_top_categorias_u4
        (actual time=0.092..0.092 rows=666.00 loops=1)
Buffers: shared hit=271
Planning Time: 0.226 ms
Execution Time: 1.280 ms
```

Auditoría y pruebas de cambios

La auditoría compara, por cada detalle, fecha, ID y nombre de categoría y subtotal contra pedido, producto y categoría. Su SELECT final devolvió 0 filas. El mismo resultado se observó tras modificar cantidad, fecha de pedido, categoría del producto y nombre de categoría dentro de una transacción de prueba. Una línea nueva obtuvo correctamente los cuatro valores; la transacción de ensayo terminó en ROLLBACK.

```
... WHERE dp.fecha_venta_u4 IS DISTINCT FROM fecha_esperada
       OR dp.categoria_id_u4 IS DISTINCT FROM categoria_esperada
       OR dp.categoria_nombre_u4 IS DISTINCT FROM nombre_esperado
       OR dp.subtotal_u4 IS DISTINCT FROM subtotal_esperado;
-- Resultado observado: 0 filas
```

Archivos entregados y orden de ejecución

Archivo	Contenido / uso
sql/tp_fnbc_control_lote.sql	Ejecutar primero en copia: extensión, migración, vista y equivalencia.
sql/tp_desnormalizacion_top_categorias.sql	Ejecutar luego: EXPLAIN antes, backfill, triggers, índice, EXPLAIN después y auditoría.
README_U4.md / documentacion/evidencia_u4.md	Instrucciones reproducibles y resultados completos de la prueba local.

Nota: el motor remoto de Railway no se modificó para esta prueba; la evidencia corresponde a la copia local poblada.